

Exercise 1: Configuring a Basic Spring Application

Step 1: Create Maven project: **LibraryManagement**

Step 2: pom.xml Dependency

```
<dependency>
<groupId>org.springframework</groupId>
<artifactId>spring-context</artifactId>
<version>5.3.30</version>
</dependency>
```

BookRepository.java

```
package com.library.repository;
public class BookRepository {
    public void showBooks(){ System.out.println("Book Repository is working."); }
}
```

BookService.java

```
package com.library.service;
import com.library.repository.BookRepository;
public class BookService {
    private BookRepository bookRepository;
    public void setBookRepository(BookRepository b){ this.bookRepository=b; }
    public void displayBooks(){
        System.out.println("Book Service is working.");
        bookRepository.showBooks();
    }
}
```

applicationContext.xml

Define bean 'bookRepository' for com.library.repository.BookRepository and bean 'bookService' for com.library.service.BookService with property bookRepository referring to bookRepository.

MainApp.java

Load ApplicationContext using ClassPathXmlApplicationContext("applicationContext.xml"), get bean 'bookService', and call displayBooks().

Output

Book Service is working.
Book Repository is working.

Explanation

Spring manages the objects (beans). applicationContext.xml defines the beans and injects BookRepository into BookService using setter injection. MainApp loads the context and runs the application.

Viva Questions

1. Spring Framework: Java framework for enterprise applications.
2. Bean: Object managed by Spring.
3. applicationContext.xml: Configuration file for beans.
4. Dependency Injection: Spring provides required objects.
5. ApplicationContext: Loads configuration and manages beans.
6. Setter Injection: Injects dependencies using setter methods.